PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
100-ft Ring Road, Bengaluru – 560085, Karnataka, India

Capstone Project Report Phase - 1

on

CleanNode- IoT based Public Sanitation System

Submitted by

ADHYA PANDEY - (PES1UG24CA004)
RISHIKA CHOWDHARY.B - (PES1UG24CA025)
CHINMAYI C GOWDA - (PES1UG24CA037)
LAVANYA.R - (PES1UG24CA081)

Jan 2026 - May 2026

Under the guidance of

Mr.Dilip Kumar Maripuri
Associate Professor
Department of Computer Applications,
PES University, Bengaluru – 560085

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER APPLICATIONS

PROGRAM - BACHELOR OF COMPUTER APPLICATIONS

Certificate

This is to certify that the project entitled

CleanNode- IoT based Public Sanitation System

is a bonafide work carried out by

ADHYA PANDEY (PES1UG24CA004)
RISHIKA CHOWDHARY.B (PES1UG24CA025)
CHINMAYI C GOWDA (PES1UG24CA037)
LAVANYA.R (PES1UG24CA081)

in partial fulfillment for the completion of **Capstone Project Phase - 1** work in the Program of Study BCA under rules and regulations of **PES University**, Bengaluru during the period **Jan 2026 - May 2026**. The project report has been approved as it satisfies the academic requirements of 4th semester BCA.

Guide	**Programme Coordinator**	**Chairperson**	**Dean -**
Mr.Dilip Kumar Maripuri	**Dr. A. Lekha**	**Dr. Veena S**	**Faculty of Engineering and Technology**
Associate Professor Dept. of Computer Applications PES University Bengaluru - 560085	Professor Dept. of Computer Applications PES University Bengaluru - 560085	Professor Dept. of Computer Applications PES University Bengaluru - 560085	PES University Bengaluru - 560085
Date :	**Date :**	**Date :**	**Date :**

Declaration

We hereby declare that the project entitled, **CleanNode- IoT based Public Sanitation System**, is an original work done by me under the guidance of **Mr.Dilip Kumar Maripuri**, Associate Professor, PES University and is being submitted in partial fulfillment of the requirements for completion of 4th Semester course work in the Program of Study BCA.

All corrections/suggestions indicated for internal assessment have been incorporated in the report.

The plagiarism check has been done for the report and is below the given threshold.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other course.

ADHYA PANDEY	**RISHIKA CHOWDHARY.B**	**CHINMAYI C GOWDA**	**LAVANYA.R**
(PES1UG24CA004)	**(PES1UG24CA025)**	**(PES1UG24CA037)**	**(PES1UG24CA081)**

Place: Bengaluru

Date : April 6, 2026

Acknowledgment

We take great pleasure in expressing our sincere gratitude to all those who have guided us and supported us to successfully complete this project.

We express our sincere gratitude to the Vice Chancellor of PES University, **Dr. J Suryaprasad**, the Chairperson **Dr. Veena S**, and the BCA Programme Coordinator **Dr. A. Lekha**, who gave us an opportunity to go ahead with this project.

We are grateful to our guide, **Mr.Dilip Kumar Maripuri**, Associate Professor, Department of Computer Applications, who has been our source of inspiration and provided us with guidance, encouragement and support during the course of the project.

ADHYA PANDEY	**RISHIKA CHOWDHARY.B**	**CHINMAYI C GOWDA**	**LAVANYA.R**
(PES1UG24CA004)	**(PES1UG24CA025)**	**(PES1UG24CA037)**	**(PES1UG24CA081)**

Abstract

Public toilets in urban areas often face poor hygiene and inefficient maintenance due to the lack of real-time monitoring systems. Manual inspection methods and fixed cleaning schedules lead to delayed responses and improper resource utilization, resulting in unsanitary conditions and reduced public usage. This highlights the need for an automated solution to improve sanitation management.

This project proposes an IoT-based smart toilet monitoring system designed to enhance hygiene and maintenance efficiency. The system focuses on collecting real-time data from various sensors to continuously monitor restroom conditions and support data-driven decision-making. By providing timely alerts and accurate information, the system ensures that cleaning activities are performed only when required, thereby improving efficiency.

The system uses an ESP32 microcontroller integrated with sensors such as MQ-135, DHT22, PIR, and water flow sensors to monitor air quality, temperature, humidity, occupancy, and water usage. The collected data is transmitted and displayed through a web-based interface developed using React.js, allowing authorities to monitor conditions and receive alerts.

The proposed system helps improve sanitation by continuously monitoring restroom conditions and identifying issues at the right time. It reduces the need for manual checking and allows cleaning to be done based on actual requirements. By providing real-time updates and alerts, it supports quicker response and better maintenance decisions. The system also makes better use of available resources and improves overall efficiency. Overall, it offers a practical solution for managing public toilet hygiene in urban areas.

Contents

1 Introduction **1**

1.1 Problem Scenario 1

1.2 Proposed Solution 1

1.3 Purpose 1

1.4 Scope 2

2 Literature Survey **3**

2.1 Domain Survey 3

2.1.1 Introduction to the Domain 3

2.1.2 Core Terminologies and Definitions 4

2.1.3 Associated Theoretical Concepts 4

2.1.4 Challenges and Open Problems 5

2.2 Literature Survey 6

2.2.1 Research Review 6

2.2.2 Summary of Reviewed Research Papers 18

2.2.3 Identified Research Gaps 18

2.2.4 Comparative Study of Existing Applications 19

3 Hardware and Software Requirements **20**

3.1 Introduction 20

3.2 Hardware Requirements 20

3.2.1 Minimum Hardware Specifications 20

3.2.2 Recommended Hardware Specifications 21

3.2.3 Sensors 21

3.3 Software Requirements 21

3.3.1 Operating System 21

3.3.2 Development Tools 21

3.3.3 Programming Languages & Frameworks 22

4 Software Requirements Specification **23**

4.1 Users 23

4.2 Functional Requirements 24

4.3 Non-Functional Requirements 27

A Writing an Abstract **30**

B Writing Introduction **32**

C Writing Literature Survey **34**

D Hardware and Software Requirements **37**

E Writing Software Requirements Specification **40**

F Writing System Design **43**

G Writing Detailed Design **46**

List of Tables

2.1 Summary of Reviewed Research Papers 18
2.2 Comparative Study of Existing Applications 19

3.1 Minimum Hardware Requirements 20
3.2 Recommended Hardware Specifications 21
3.3 Sensors Used 21
3.4 Operating System Requirements 22
3.5 Development Tools 22
3.6 Programming Languages and Frameworks 22

List of Figures

List of Algorithms

Listings

Chapter 1

Introduction

1.1 Problem Scenario

Public toilets in urban areas such as Bengaluru suffer from poor hygiene and inefficient maintenance due to the absence of real-time monitoring systems. Authorities rely on manual inspections and fixed cleaning schedules, which lead to delayed responses and improper resource utilization. Critical issues such as poor air quality, water failure, and overcrowding are not detected in time, and users have no access to information about toilet availability or cleanliness. Existing complaint systems are slow and ineffective, highlighting the need for an automated and intelligent solution to improve sanitation management.

1.2 Proposed Solution

To address these issues, an IoT-based smart toilet monitoring system is proposed that uses multiple sensors to collect real-time data on parameters such as air quality, temperature, humidity, occupancy, and water usage. The system processes this data to evaluate overall cleanliness and detect abnormal conditions, enabling timely maintenance. A web-based interface is provided to display live status, cleanliness levels, and alerts for authorities, while also allowing users to access information about toilet availability. This approach ensures efficient resource utilization, proactive cleaning, and improved sanitation management.

1.3 Purpose

The purpose of this project is to develop an IoT-based smart toilet monitoring system that improves sanitation management through real-time monitoring and automated decision-making. The system aims to continuously collect data from sensors to track parameters such as air quality, temperature, humidity, occupancy, and water usage. Based on this data, it helps in identifying cleanliness levels and detecting abnormal conditions at an early stage. The project also focuses on reducing manual inspection, improving response time for maintenance, and ensuring efficient utilization of cleaning resources. Overall, it aims to enhance hygiene standards and provide a more reliable and efficient solution for public toilet management.

1.4 Scope

This project focuses on developing an IoT-based smart toilet monitoring system for real-time sanitation management. It monitors parameters such as air quality, temperature, humidity, occupancy, and water usage using sensors connected to a microcontroller. The system provides real-time data and alerts through a web-based interface to support timely cleaning and maintenance decisions. It is designed for use in public toilets in urban areas to improve hygiene and resource utilization. The system is limited to monitoring and alert functionalities, and features like mobile applications or large-scale deployment can be considered in future work.

Chapter 2

Literature Survey

2.1 Domain Survey

2.1.1 Introduction to the Domain

The Internet of Things (IoT) has fundamentally changed the way physical environments are monitored and managed. By connecting everyday objects to the internet through embedded sensors and communication modules, IoT systems enable real-time data collection, remote monitoring, and automated decision-making without requiring constant human intervention. Over the past decade, IoT-based monitoring has expanded across sectors including healthcare, agriculture, industrial automation, and urban infrastructure, proving itself as a reliable and scalable approach to solving real-world problems.

Within urban infrastructure, IoT monitoring systems have found particular relevance in managing public facilities and services. Traditional methods of maintaining public spaces relied heavily on fixed schedules and manual inspection, which are both resource-intensive and reactive in nature. Issues would often go undetected until they became serious, leading to poor service quality and public dissatisfaction. IoT-based systems address this by enabling continuous, sensor-driven monitoring that can detect anomalies, generate alerts, and provide actionable data to administrators in real time, shifting the approach from reactive to proactive management.

Despite the growing adoption of IoT in various domains, its application in public sanitation monitoring remains limited and fragmented. Most existing systems either rely on single-parameter sensing, lack cloud integration, or do not provide a unified dashboard for city-level oversight. There is a clear need for a more comprehensive, multi-sensor IoT monitoring system specifically designed for public sanitation infrastructure — one that can track occupancy, air quality, water usage, and cleanliness indicators simultaneously, and communicate this data to the relevant authorities through an intelligent, centralized platform.

2.1.2 Core Terminologies and Definitions

- **Internet of Things (IoT)** – A network of physical devices embedded with sensors and connectivity that collect and exchange data over the internet automatically, without human involvement.
- **Smart Sanitation** – The use of IoT sensors, data analytics, and automated alerts to monitor and manage public sanitation facilities in real time, moving away from manual inspection-based approaches.
- **Real-Time Monitoring** – Continuous collection and transmission of sensor data to provide immediate visibility into the current condition of a facility, enabling faster response to hygiene or maintenance issues.
- **Predictive Cleanliness Index** – A composite score ranging from 0 to 100, computed by an ML model using multi-sensor inputs such as ammonia levels, occupancy count, humidity, and flush health, representing the real-time cleanliness state of a toilet facility.
- **Flush Health Monitoring** – A practice of tracking water flow activity post-occupancy to detect whether a flush occurred and whether flow rate falls within the normal range, enabling early identification of blockages or maintenance needs.
- **Usage Pattern Learning** –A data mining approach applied to historical occupancy time-series data to identify recurring traffic trends such as peak hours and day-of-week patterns, enabling proactive and schedule-based cleaning decisions.
- **Alert Notification System** – An automated mechanism that sends real-time alerts via SMS or push notifications to concerned authorities when sensor readings cross defined thresholds.

2.1.3 Associated Theoretical Concepts

- **Publish-Subscribe Model** – Ensures sensor data from each NodeMCU is efficiently transmitted to AWS IoT Core via MQTT, where the backend subscribes to relevant topics for processing.
- **Multi-Sensor Data Fusion** – Combines readings from PIR, MQ-135, DHT22, and YF-S201 sensors to compute a unified Predictive Cleanliness Index (PCI) for each toilet facility.
- **Machine Learning for Predictive Analysis** – Used to analyze historical sensor data and occupancy trends in order to compute real-time PCI scores and generate proactive cleaning schedules.
- **Threshold-Based Alerting** – Triggers automated SMS and push notifications to BBMP supervisors when sensor readings exceed predefined cleanliness or safety limits.

- **RESTful Architecture** – Enables standardized communication between CleanNode's backend, React.js dashboard, and Flutter mobile app through HTTP-based APIs.

2.1.4 Challenges and Open Problems

- **Scalability** – Deploying and managing hundreds of IoT sensor nodes across a city like Bengaluru generates a massive continuous stream of data, making it difficult for existing systems to maintain real-time performance without significant infrastructure overhead.
- **Real-Time Adaptability** – Most current sanitation monitoring approaches rely on fixed cleaning schedules and cannot dynamically respond to sudden changes in toilet usage or unexpected sensor anomalies.
- **Hardware Reliability** – Public toilet environments are humid, chemically exposed, and prone to vandalism, making long-term sensor reliability a persistent challenge that most research prototypes do not adequately address.

2.2 Literature Survey

2.2.1 Research Review

1. **[1]P. Deshmukh, Abhishek Mohite, Harshal Bhoir, R. Patil, and Aparna Bhonde, "Intelligent Public Toilet Monitoring System Using IoT," 2020 IEEE Bangalore Humanitarian Technology Conference (B-HTC), Oct. 2020, doi: https://doi.org/10.1109/b-htc50970.2020.9297839.**
   Available: IEEEXplore.

   - **Abstracted Summary**

     Manual monitoring of public toilet hygiene is time-consuming and often inefficient, leading to poor maintenance and delayed cleaning. To address this issue, an IoT-based monitoring system is proposed that uses sensors to track parameters such as odor levels, water usage, and occupancy. The system provides real-time alerts when cleaning is required, reducing the need for manual inspection. This approach improves hygiene standards and ensures timely maintenance.

   - **Problem Statement**

     Municipal bodies in India lack a centralized system to monitor the cleanliness of public toilets or to track the quality of the cleaning carried out by assigned staff. Toilets need to be visited manually to verify their hygiene conditions, resulting in significant time and manpower wastage. Despite the Swachh Bharat Mission reporting over 4.9 lakh community and public toilets in working condition in urban India, around 44 percent of the population still defecates in the open due to poorly maintained facilities.

   - **Proposed Solution**

     The system deploys a NodeMCU ESP8266 microcontroller with multiple sensors at each toilet unit — an MQ-137 ammonia gas sensor for air quality detection, an ultrasonic sensor for visitor counting and water level monitoring, an RFID scanner for cleaner attendance tracking, an IR sensor for soap availability, an LDR sensor for luminosity measurement, and a water sensor for clog detection. All sensor data is transmitted to Google Firebase at five-minute intervals via Wi-Fi. A ReactJS dashboard provides centralized visualization, and a React Native mobile app sends push notifications to cleaners when conditions degrade.

   - **Methodology**

     Sensor data collected from IoT devices is pre-processed to remove anomalies such as gaps and false readings, normalized, and stored in Firebase with timestamps. An ARIMA (Auto-Regressive Integrated Moving Average) model is applied on the time series sensor data to forecast future values for the entire day. The model's performance is evaluated using Mean Forecast Error (MFE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE).

An RFID-based attendance system tracks whether cleaners have physically visited assigned toilets, and a user feedback rating system allows toilet users to rate cleanliness via physical buttons installed at each unit.

- **Results**

  The system achieved satisfactory forecasting accuracy across different sensor parameters, enabling cleaners to take preventive action before toilet conditions deteriorated. The mobile application eliminated the need for physical inspection visits, while push notifications ensured timely cleaning. The overall system was reported as cost-effective and portable, with the IoT device being easy to deploy.

- **Conclusion**

  The proposed system provides an efficient solution for maintaining public sanitation using real-time monitoring. It reduces manual effort and ensures better cleanliness management. The system is reliable and suitable for smart city applications. However, further enhancements can improve scalability and sensor performance.

2. **[2]F. I. Azman, N. L. Saleh, F. Hashim, A. Sali, A. M. Ali, and A. S. M. Noor, "An IoT-Based Hygiene Monitoring System in the Restroom," IEEE Access, vol. 13, pp. 119348–119361, 2025, doi: https://doi.org/10.1109/access.2025.3586117.** Available: IEEEXplore.

   - **Abstracted Summary**

     This paper presents an IoT-based restroom hygiene monitoring system that optimises cleaning operations by dispatching staff only when hygiene thresholds are exceeded, particularly during high ammonia levels and poor Indoor Air Quality (IAQ). The system integrates sensors such as MQ-137 and BME680 with an ESP32 microcontroller to monitor environmental conditions and occupancy. Data is transmitted via MQTT over Wi-Fi and visualised using a Grafana dashboard for real-time monitoring and analysis. The approach enhances efficiency, reduces unnecessary cleaning, and supports data-driven hygiene management.

   - **Problem Statement**

     Traditional restroom maintenance approaches rely on manual labour and fixed cleaning schedules, which are often inefficient and costly, requiring janitors regardless of actual conditions. Public restrooms face persistent cleanliness and maintenance issues, with poor sanitation posing serious health risks including respiratory irritation, headaches, and exposure to harmful pollutants such as ammonia gas.

   - **Proposed Solution**

     The system monitors ammonia concentration using an MQ-137 sensor and analyses IAQ through humidity and gas resistance metrics via a BME680 sensor.

Alerts are triggered through Telegram notifications when ammonia exceeds 5 ppm or IAQ surpasses 100 ppm, prompting immediate cleaning action. An IR-based people counter tracks occupancy and triggers cleaning notifications every 50 users. The system includes a nightly counter reset feature at 10:00 PM coinciding with restroom closure.

- **Methodology**

  A five-month focused analysis within a two-year monitoring period (January 2023 to December 2024) was conducted. Statistical analysis was performed to study the relationship between environmental factors such as temperature, humidity, and ammonia levels.. The system was deployed with waterproof and vandalism-proof casing at 120 cm height to optimally detect ammonia, which is lighter than air.

- **Results**

  The analysis revealed ammonia concentration spikes ranging from 0 ppm to 13 $\times$ 10^{12} ppm, linked to sewer issues and cleaning without detergent. A strong inverse correlation (r = 0.91) was found between humidity-temperature and ammonia levels. Poor IAQ levels up to 466 $\times$ 10^{3} ppm were associated with excessive use of detergents and air fresheners. Despite 274 users recorded in a single day, effective ventilation maintained safe ammonia levels.

- **Conclusion**

  The study demonstrated that gas sensors and IAQ monitoring alone are insufficient for detecting physical cleanliness — the system could only assess air quality, not whether the toilet surface was visually dirty or whether the flush functioned properly. There was no multi-sensor fusion to compute a composite cleanliness score, no proactive scheduling based on usage patterns, no citizen-facing application, and no flush health monitoring mechanism.

3. **[3]A. Lokman, R. K. Ramasamy, and C.-Y. Ting, "Scheduling and Predictive Maintenance for Smart Toilet," IEEE Access, vol. 11, pp. 17983–17999, 2023, doi: https://doi.org/10.1109/access.2023.3241942.**
   Available: IEEEXplore.

   - **Abstracted Summary**

     This paper proposes an IoT-based smart toilet management system with predictive maintenance and janitor scheduling capabilities. The system, called ARIGA, combines an ARIMA model for predicting equipment degradation (Remaining Useful Life) with a Genetic Algorithm for optimising janitorial cleaning schedules based on usage patterns.

   - **Problem Statement**

High-rise buildings incur expensive cleaning costs, and cleaning schedules are typically fixed regardless of actual toilet usage patterns. Different floors have different usage patterns based on building occupancy, yet janitors are assigned uniformly. Existing systems lack usage-pattern-based proactive scheduling to efficiently allocate cleaning staff.

- **Proposed Solution**

  The system deploys infrared sensors and temperature-humidity sensors on a Raspberry Pi and TTGO ESP32 microcontrollers to collect toilet usage data. An ARIMA model predicts equipment degradation to determine when maintenance is needed.A Genetic Algorithm generates optimised janitor schedules considering various constraints. Simulation techniques were used to model user behaviour and usage patterns of the system.

- **Methodology**

  Sensor data was collected from toilets at Multimedia University's Faculty of Computing and Informatics. Time series data was split 70 percent for training and 30 percent for testing. The ARIGA model was compared against LSTM and a Greedy scheduling algorithm as baselines. Performance was evaluated using RMSE, MAE, and MAPE metrics. The scheduling algorithm was tested across scenarios with varying floors (10, 20, 30) and user populations.

- **Results**

  The ARIGA model achieved lower RMSE, MAE, and MAPE than the LSTM model, demonstrating better fitting and stability for time series prediction. The proposed scheduling algorithm reduced the number of required cleaning staff by 24.7 percentage on average and 15.0 percentage overall compared to baseline scheduling. The ARIGA model scheduled most cleaning during off-peak hours while not exceeding janitor capacity thresholds.

- **Conclusion**

  The proposed system improves maintenance planning by predicting failures in advance. It reduces downtime and enhances operational efficiency. The use of predictive models ensures better scheduling decisions. However, further improvements are required to enhance prediction accuracy.

4. [4]**T. Stephan, Krishna, C.-C. Lin, Ujjwal Sumesh, S. Agarwal, and H. Kim, "ProWaste for proactive urban waste management using IoT and machine learning," Scientific Reports, vol. 15, no. 1, Jul. 2025, doi: https://doi.org/10.1038/s41025-08452-x.**
Available:Springer Nature (Scientific Reports).

   - **Abstracted Summary**

     This paper introduces ProWaste, an end-to-end IoT and machine learning platform that proactively prioritises urban Waste Collection Centre (WCC) servicing in Bengaluru, India. The system fuses 15 automated and manual indicators to predict daily WCC criticality and pushes mobile alerts to maintenance crews via a Flutter-based mobile application.

   - **Problem Statement**

     Urban waste collection centres routinely overflow because maintenance routes are scheduled reactively rather than on data-driven forecasts. Prevailing systems intervene only after overflow events, many models ignore influential factors such as proximity to activity hubs and weather dynamics, and few investigations report end-to-end field deployments.

   - **Proposed Solution**

     ProWaste streams data from low-cost sensors, public APIs, and a mobile application to a Google Firebase cloud database. Fifteen input features including population density, weather conditions, maintenance history, and weekly waste build-up are fed into a machine learning classification module. A Sustainable Smart Waste Management (SSWM) Flutter app alerts field teams and dynamically reorders maintenance queues based on predicted criticality scores.

   - **Methodology**

     Twenty-five off-the-shelf classifiers were benchmarked under repeated stratified cross-validation. Binary Particle Swarm Optimisation (BPSO) was used for feature selection, removing 80 percent of input features. Feature selection techniques were used to identify the most important factors affecting waste accumulation, and data analysis methods were applied to understand how each factor influences the prediction results. The system was validated on 6,954 daily records from 57 Bengaluru WCCs over four months.

   - **Results**

     A Decision Tree Classifier offered the best balance of interpretability and near-top accuracy. The compact three-feature model predicted criticality with greater than 99 percent accuracy on a hold-out test set. SHAP analysis confirmed interpretability of the reduced model. The system demonstrated reliable real-time operation and measurable reductions in unnecessary collections.

   - **Conclusion**

The system enhances waste management by using predictive analytics for efficient collection. It reduces operational costs and improves service quality. The approach is scalable and suitable for smart cities. Future improvements can focus on integrating more real-time data sources.

5. **[5]Q. P. Ha, S. Metia, and M. D. Phung, "Sensing Data Fusion for Enhanced Indoor Air Quality Monitoring," IEEE Sensors Journal, vol. 20, no. 8, pp. 4430–4441, Apr. 2020, doi: https://doi.org/10.1109/jsen.2020.2964396.**
Available:IEEEXplore.

   - **Abstracted Summary**

     This paper proposes a multi-sensor data fusion approach for indoor air quality monitoring in smart buildings. The system merges an Indoor Air Quality Index (IAQI) and a humidex into an Enhanced Indoor Air Quality Index (EIAQI) using an Extended Fractional-Order Kalman Filter (EFKF) for accurate real-time prediction.

   - **Problem Statement**

     Indoor air quality is affected by numerous pollutants including CO, CO, NH, HS, toluene, and ethanol, as well as temperature and humidity. Individual sensor readings may contain noise, missing data, and nonlinearities, making single-sensor monitoring unreliable for accurately assessing overall air quality conditions.

   - **Proposed Solution**

     The system utilises a network of Waspmote sensors measuring eight air pollutants (CO, CO, H, NH, CHO, HS, CH, O) along with temperature and humidity. An IAQI is calculated using linear interpolation formulas based on pollutant breakpoints. A humidex value is computed from temperature and humidity data. These are fused into an EIAQI using weighted combination with an EFKF to handle nonlinearity and missing measurements.

   - **Methodology**

     The system was deployed in an office building at the University of Technology Sydney with over 100 Waspmote sensors. A case study involving a fainting student incident on 24th August 2016 was analysed. EFKF with Matérn covariance function was used for spatial distribution modelling. Performance was evaluated using MAPE, RMSE, and R^2 metrics.

   - **Results**

     The EFKF-based prediction achieved MAPE values between 0.3 and 0.9 across different pollutants. The fused EIAQI accurately reflected the poor air quality conditions during the incident day, revealing that the cause was exposure to unhealthy levels of CO and toluene rather than low oxygen. The composite index

approach proved more informative than monitoring individual sensor readings in isolation.

- **Conclusion**

  Combining multiple sensor data improves accuracy in air quality monitoring. It reduces uncertainty and enhances reliability of measurements. The approach is useful for real-time environmental assessment. Further optimization can improve performance efficiency.

6. **[6]M. Vibhute, J. Biju, K. Shelar, N. Koli, and O. Gubyad, "SMART DRAINAGE MONITORING SYSTEM," International Journal For Multidisciplinary Research, vol. 7, no. 2, Apr. 2025, doi: https://doi.org/10.36948/ijfmr.2025.v07i02.40325**

   Available:IJFMR.

   - **Abstracted Summary**

     This paper proposes a smart drainage monitoring system that combines a YF-S201 flow sensor, a DS18B20 temperature sensor, and a gas sensor to detect anomalies in water flow within drainage pipelines. The system uses an Arduino microcontroller with GSM-based SMS alerts for real-time blockage detection.

   - **Problem Statement**

     Drainage systems are essential to urban infrastructure, but blockages can result in waterlogging, structural damage, and environmental hazards. Conventional monitoring relies on manual examination, which is labour-intensive, time-consuming, and frequently unsuccessful in identifying obstructions before they become serious problems.

   - **Proposed Solution**

     The YF-S201 flow sensor is deployed in a loop configuration with a non-return valve to indirectly detect blockages — when flow rate falls below a threshold, a possible obstruction is indicated. A DS18B20 temperature sensor monitors pipe temperature to detect overheating or damage. A gas sensor (MQ-135) detects hazardous gas accumulation. An Arduino Uno processes sensor data and displays readings on an LCD, while a SIM900A GSM module sends SMS alerts to authorities.

   - **Methodology**

     The system was designed for deployment in urban drainage networks, industrial pipelines, and flood control systems. The YF-S201 sensor uses a Hall-effect mechanism where a rotor with embedded magnets spins in response to fluid flow, and the Hall sensor detects changes in magnetic field corresponding to flow rate changes. Data is transmitted to ThingSpeak cloud platform via GPRS for remote monitoring.

- **Results**

  The system successfully demonstrated real-time blockage detection through flow rate monitoring, temperature anomaly detection, and hazardous gas sensing. SMS alerts were triggered promptly when conditions exceeded safety thresholds. The system was reported as cost-effective and scalable for multiple deployment scenarios.

- **Conclusion**

  Early detection of drainage issues helps prevent waterlogging and infrastructure damage. It reduces the need for continuous manual monitoring. The approach is practical for urban infrastructure management. Future improvements can focus on large-scale deployment.

7. **[7]H. Zhang, R. Zhang, and J. Sun, "Developing real-time IoT-based public safety alert and emergency response systems," Scientific Reports, vol. 15, no. 1, pp. 1–25, Aug. 2025, doi: https://doi.org/10.1038/s41598-025-13465-7.**
Available:Springer Nature (Scientific Reports).

   - **Abstracted Summary**

     This paper presents the design and evaluation of a real-time IoT-based emergency response and public safety alert system for rapid detection, classification, and dissemination of alerts during critical incidents. The system combines distributed heterogeneous sensors, edge computing nodes, and cloud platforms with a dashboard and notification infrastructure.

   - **Problem Statement**

     Many current public safety systems remain encumbered by outdated protocols, manual processes, and fragmented communication channels. Emergency response frameworks are predominantly reactive, relying on eyewitness reports or distress calls to initiate action. Existing systems lack real-time data integration, dynamic situational awareness, and inter-agency coordination capabilities.

   - **Proposed Solution**

     The architecture combines a distributed network of heterogeneous sensors (gas, flame, vibration, biometric), edge computing nodes (Raspberry Pi, ESP32), and cloud platforms (AWS IoT, Firebase). Communication uses secure MQTT over TLS with fallback to LoRa for low-connectivity environments. A cloud-based dashboard provides real-time visualisation, and push notifications via Firebase Cloud Messaging (FCM) and SMS alerts reach emergency responders and citizens.

   - **Methodology**

     A prototype was implemented and tested across four emergency scenarios — fire, traffic accident, gas leak, and medical distress — within a smart city simu-

lation testbed. Performance was evaluated on alert latency, detection accuracy, scalability, reliability, and uptime. A comprehensive comparison against seven state-of-the-art systems was conducted.

- **Results**

  The system achieved alert latency under 450 ms, detection accuracy exceeding 95 percentage, scalability supporting over 12,000 concurrent devices, 99.1 percentage alert success rate, and 99.8 percentage uptime. The comparison against existing systems confirmed superior performance in latency, reliability, and scalability.

- **Conclusion**

  Real-time alert mechanisms improve response time during emergency situations. It enhances communication and ensures quick action. The approach supports large-scale smart city applications. Future enhancements can include predictive analytics.

8. **[8]K. Dineva and T. Atanasova, "Design of Scalable IoT Architecture Based on AWS for Smart Livestock," Animals, vol. 11, no. 9, p. 2697, Sep. 2021, doi: https://doi.org/10.3390/ani11092697.**
   Available:MDPI.

   - **Abstracted Summary**

     This paper presents a specially designed cloud-based architecture built on Amazon Web Services (AWS) for monitoring livestock using cyber-physical systems (CPS) and IoT equipment. The architecture features automated scaling, comprehensive security, and a wide range of cloud-native services for data ingestion, processing, and analytics.

   - **Problem Statement**

     Due to the growing number of connected IoT devices, the scalability capacity and available computing power of existing architectural frameworks are being reached. Multi-layer and microservice architectures face limitations in handling large volumes of heterogeneous data from numerous IoT devices simultaneously. Most current livestock systems lack centralised IoT device management, remote configuration, and security patch deployment.

   - **Proposed Solution**

     The architecture is built entirely on AWS cloud infrastructure using services including AWS IoT Core for device connectivity, streaming data services for real-time ingestion, data lake services for storage, machine learning services for predictive analytics, and notification services for alerting farmers. The system follows the five pillars of AWS Well-Architected Framework: Operational Excellence, Security, Reliability, Performance Efficiency, and Cost Optimisation.

- **Methodology**

  The system was developed following Agile (Scrum) methodology with six stages: requirements analysis, planning, architecture design, software development, architecture testing, and deployment monitoring. A stress test was performed to evaluate the architecture's data processing capabilities, including throughput ingestion rates, serverless function performance, total average latency during high loads, and error counts during continuous operation.

- **Results**

  Experimental results showed that the proposed architecture with AWS services was fully capable of processing the required amount of data and allowed the CPS-IoT infrastructure to use automated scaling mechanisms. The system supported up to 10,000 IoT devices per second with minimal latency and high resiliency through distributed storage and processing.

- **Conclusion**

  Cloud-based architecture improves scalability and efficient data handling. It supports real-time monitoring and automation in large-scale environments. The implementation enhances productivity in smart farming. Further improvements can focus on advanced analytics.

9. **[9]K. Pardini, J. J. P. C. Rodrigues, O. Diallo, A. K. Das, V. H. C. de Albuquerque, and S. A. Kozlov, "A Smart Waste Management Solution Geared Towards Citizens," Sensors, vol. 20, no. 8, p. 2380, Apr. 2020, doi: https://doi.org/10.3390/s20082380.**
   Available:MDPI.

   - **Abstracted Summary**

     This paper proposes an IoT-based smart waste management solution that includes a real prototype of a smart container, a waste management web and mobile application, and integration through an IoT middleware platform. The system focuses on a citizen-centric approach where residents can access information about public waste bins through a mobile application.

   - **Problem Statement**

     The waste management system currently used in cities follows an old and outdated model that no longer meets the needs of municipalities. It is inefficient, practiced through large fleets of collection trucks travelling long distances with fixed schedules, causing unnecessary costs, time wastage, and environmental damage through fuel emissions and poor waste handling. Few existing solutions are focused on the citizen perspective or provide mobile-based solutions for usability.

   - **Proposed Solution**

The smart bin prototype is equipped with ultrasonic sensors for fill-level detection in multiple compartments (glass, paper, plastic, organic waste). Sensor data is transmitted via wireless communication to an IoT middleware (In.IoT) for storage and processing. A web application and mobile application allow citizens to check the availability and fill level of waste compartments scattered across a city, and to locate the nearest available bin. Optimised collection routes are generated for waste collection trucks.

- **Methodology**

  The system follows a three-layer IoT architecture: perception layer (sensors), network layer (communication), and application layer (user interfaces and analytics). A real prototype was created, deployed, and validated through real-scale experimentation. The In.IoT middleware provided data storage, processing, and API services for both the citizen application and the collection management system.

- **Results**

  The real-scale experiment demonstrated that the proposed system could efficiently change the way people deal with garbage and optimise economic and material resources. The citizen mobile application provided real-time visibility of bin status, and the route optimisation reduced unnecessary truck trips. The system was validated as ready for real-world deployment.

- **Conclusion**

  Real-time waste monitoring improves collection efficiency and reduces operational challenges. It supports better resource utilization and service quality. The approach is suitable for smart city applications. Future work can focus on automation and user interaction.

10. **[10]M. Romano, P. Díaz, and I. Aedo, "Gamification-less: may gamification really foster civic participation? A controlled field experiment," Journal of Ambient Intelligence and Humanized Computing, Jun. 2021, doi: https://doi.org/10.1007/s12652-021-03322-6.**
    Available:Springer.

    - **Abstracted Summary**

      This paper presents a between-group design experiment studying the effects of gamification on civic participation using two mobile applications — one gamified and one without game elements. The study is grounded in Self-Determination Theory (SDT) and evaluates whether gamified systems foster better civic engagement than non-gamified ones.

    - **Problem Statement**

In the context of smart communities, active and continuous collaboration between citizens, organisations, and institutions is essential. However, mobile platforms for civic participation typically suffer from low engagement levels, not achieving the purpose for which they were created. Motivation to participate cannot be taken for granted and needs to be actively fostered through appropriate design strategies.

- **Proposed Solution**

  The gamified mobile application (iWarn) includes game elements mapped to SDT intrinsic motivators: points for completing missions (Competence), rankings and team comparisons (Relatedness), and progressive access to more difficult missions based on user level (Autonomy). The application enables citizens to collect information on potential risks through missions, with achievements tracked through a ranking system. A non-gamified version with identical functionality but without game elements was created for comparison.

- **Methodology**

  A between-subjects controlled field experiment was conducted with 20 participants divided into two groups for one week. Group 1 used the gamified app (GApp) and Group 2 used the non-gamified app (NgApp). Engagement was measured through a questionnaire based on O'Brien and Toms's User Engagement Scale (Perceived Usability, Felt Involvement, Aesthetics, Focused Attention) and quantitative system logs (number of logins, uploaded pictures, accomplished missions).

- **Results**

  The gamified application generated a better user experience and civic engagement across all measured dimensions. All gamification elements were positively evaluated by users, with participants particularly appreciating the ability to interact and compete with other users and the progression system for accessing more difficult missions. Gamified users showed higher numbers of logins, missions accomplished, and content uploaded compared to the non-gamified group.

- **Conclusion**

  Gamification techniques increase user engagement and participation effectively. It motivates users through interactive design elements. The study shows positive impact on civic involvement. Further research can explore wider applications.

2.2.2 Summary of Reviewed Research Papers

Paper	Authors & Year	Problem Statement	Methodology	Key Findings
Intelligent Public Toilet Monitoring System Using IoT	P. Deshmukh,Abhishek Mohite,Harshal Bhoir,Rushikesh Patil,Aparna Bhonde, 2020	Manual monitoring is time-consuming and inefficient	IoT sensors with real-time monitoring and prediction	Improves hygiene and reduces manual effort
IoT-Based Hygiene Monitoring System in Restroom	F. I. Azman, N.L. Saleh,F.Hashim,A.Sali,A.Mulati and A.S.M.Noor, 2025	Fixed cleaning schedules lead to poor hygiene	Sensor-based monitoring with data analysis	Enables condition-based cleaning
Scheduling and Predictive Maintenance for Smart Toilet	A. Lokman,R.Kanesaraj Ramaswamy , 2023	Lack of usage-based scheduling increases cost	Predictive models with scheduling techniques	Improves planning and reduces cost
ProWaste- Urban Waste Management Using IoT and ML	T. Stephan,S.M.Hari Krishna,Chia-Chen Lin,Ujjwal Sumesh,Saurabh Agarwal and Kim Hyunsung, 2025	Reactive waste collection leads to overflow issues	IoT with machine learning techniques	Improves waste collection efficiency
Indoor Air Quality Monitoring using Sensor Fusion	Q. P. Ha,S. Metia, and M. D. Phung, 2020	Single sensor data is unreliable	Multi-sensor data fusion techniques	Improves monitoring accuracy
Smart Drainage Monitoring System	M. Vibhute et al., 2025	Manual inspection fails to detect blockages	Sensor-based monitoring with alert system	Enables early detection
IoT-Based Emergency Alert System	Han Zhang,Runze Zhang, 2025	Delay in emergency response	Real-time monitoring with alerts	Improves response time
AWS-Based IoT Architecture for Livestock	Kristina Dineva and Tatiana Atanasova, 2021	Existing systems lack scalability	Cloud-based IoT architecture	Supports large-scale systems
Smart Waste Management Solution	Kellow Pardini,Joel J.P.C Rodrigues, 2020	Traditional waste collection is inefficient	IoT monitoring with mobile applications	Optimizes resource usage
Gamification for Civic Participation	Macro Romano,Paloma Diaz,Ignacio Aedo, 2021	Low user engagement in systems	Gamification-based application approach	Increases user participation

Table 2.1: Summary of Reviewed Research Papers

2.2.3 Identified Research Gaps

- **No Flush Health Monitoring** – Current solutions do not include any mechanism to check whether the toilet flush is functioning properly. Flush failure is a common issue affecting cleanliness, yet it is not considered in existing systems.
- **No Composite Cleanliness Score** – Most existing systems monitor parameters such as ammonia, humidity, or temperature individually. However, they do not combine these factors to provide a single overall cleanliness score. This makes it difficult to quickly assess the hygiene condition of a restroom.
- **Lack of Usage-Based Scheduling** – Many systems either follow fixed cleaning schedules or react only after conditions worsen. They do not use real-time usage data to schedule cleaning proactively, which reduces efficiency.
- **Limited Scalability** – Several proposed systems are designed for small-scale implementation and may not perform efficiently when deployed across large areas.

- **No Emergency Alert Mechanism** – Most systems do not handle critical situations such as gas leakage, flooding, or unusual occupancy. Real-time alert mechanisms for such emergencies are not adequately addressed.
- **Lack of User Engagement Techniques** – None of the existing solutions use techniques like gamification to encourage user participation. This limits user involvement in maintaining and reporting cleanliness.
- **No Citizen-Facing Application** – Existing systems are mainly designed for authorities and maintenance staff. There is no provision for users to locate nearby clean restrooms or provide feedback on hygiene conditions.

2.2.4 Comparative Study of Existing Applications

Application	Features	Performance	User Experience	Limitations
Swachhata-MoHUA App	Complaint-based reporting, GPS tagging, photo upload	Works well for large-scale issue reporting	Simple and easy for citizens to use	Reactive system, no automatic monitoring or verification
Indore Smart Sanitation Dashboard	IoT sensors, real-time alerts, AI-based prediction	High efficiency in monitoring multiple toilets	Dashboard-based monitoring for authorities	Expensive, proprietary, not easily scalable
NAMMA TOILET	Multi-sensor monitoring, RFID tracking, data analytics	Conceptually strong with multiple features	Limited real-world usage	Not deployed in real environment
IAQ-Based Monitoring Systems	Gas sensors, air quality detection	Detects odor and pollution effectively	Basic alert-based interaction	Cannot measure actual cleanliness
Predictive Cleaning Systems	Usage-based prediction, scheduling algorithms	Improves cleaning efficiency	Limited user interaction	No real-time monitoring integration
Citizen Feedback Apps	Issue reporting, feedback system	Useful for complaint tracking	Easy for users to report problems	No automation, depends on user input

Table 2.2: Comparative Study of Existing Applications

Chapter 3

Hardware and Software Requirements

3.1 Introduction

This chapter describes the **hardware and software requirements** used for the development and implementation of the smart toilet monitoring system. The selected technologies are based on factors such as **performance, scalability, cost-effectiveness, and compatibility** with IoT-based applications. The system integrates **sensors, microcontrollers**, and web technologies to ensure efficient real-time monitoring and data processing.

3.2 Hardware Requirements

A system with enough processing power is needed for the project in order to provide effective development, testing, and execution.

3.2.1 Minimum Hardware Specifications

The following table lists the **minimum hardware configuration** required:

Component	Specification
Processor	Intel Core i5 or equivalent
RAM	4GB or more
Storage	64GB or more
Microcontroller	ESP32 DevKit v1
Sensors	MQ-135, DHT22, PIR, Water Flow Sensor

Table 3.1: Minimum Hardware Requirements

3.2.2 Recommended Hardware Specifications

For optimal performance, the **recommended configuration** is:

Component	Specification
Processor	Intel Core i5 / Ryzen 5
RAM	8GB or higher
Storage	128GB or higher
Microcontroller	ESP32 DevKit v1
Sensors	MQ-135, DHT22, PIR, Water Flow Sensor(YF-S201),Reed Switch
Power Supply	5V 2A Adapter
Additional Components	10k Resistor

Table 3.2: Recommended Hardware Specifications

3.2.3 Sensors

These components monitor environmental conditions, occupancy, and system functionality.

Sensors	Description
ESP32 DevKit v1	Microcontroller with built-in Wi-Fi, operates at 3.3V
MQ-135 Gas Sensor	Detects gases like ammonia; operates at 5V
DHT22 Sensor	Measures temperature (40°C to 80°C) and humidity
PIR Sensor (HC-SR501)	Detects motion within 5–7 meters range
Reed Switch	Magnetic sensor used for door status detection
Water Flow Sensor (YF-S201)	Measures water flow and detects flush activity
10k Resistor	Used for signal stabilization
5V Power Supply	Provides power to all components

Table 3.3: Sensors Used

3.3 Software Requirements

This project's software stack consists of **development tools, programming languages, and frameworks** to guarantee seamless execution and implementation.

3.3.1 Operating System

The project was developed and tested on multiple platforms to ensure cross-platform compatibility.

3.3.2 Development Tools

To facilitate efficient coding and debugging, the following development tools were used:

Operating System	Version
Windows	Windows 10/11 (64-bit)
Linux	Ubuntu 20.04+ / Fedora 36+
macOS (Optional)	macOS 11+

Table 3.4: Operating System Requirements

Software	Purpose
Arduino IDE v2.3.2	Programming ESP32
Visual Studio Code	Code editor / IDE
Git & GitHub/GitLab	Version control & collaboration
Postman v11.89	API Testing

Table 3.5: Development Tools

3.3.3 Programming Languages & Frameworks

The project was implemented using a combination of programming languages and frameworks:

Technology	Usage
C/C++	ESP32 programming
Python (Flask v3.0+)	Backend development
Paho-mqtt v1.6.1	MQTT client handling
React.js v18.2.0	Frontend development
MongoDB	Database management
MQTT Protocol v3.1.1	Data communication
Mosquitto Broker	Message handling

Table 3.6: Programming Languages and Frameworks

Chapter 4

Software Requirements Specification

4.1 Users

- **Municipal Authority (Administrator)**

  Municipal authorities are responsible for monitoring and managing public toilet facilities using the system. They oversee sanitation conditions by viewing real-time sensor data such as air quality, occupancy, and water usage. The system helps them identify issues quickly and take appropriate actions to maintain hygiene standards.

  To ensure efficient management, the system provides alerts for abnormal conditions and displays cleanliness status through a dashboard. This allows authorities to make informed decisions regarding maintenance and resource allocation. In cities like Bengaluru, this role can be handled by organizations such as BBMP.

- **Maintenance Staff**

  Maintenance staff are responsible for cleaning and maintaining the restroom facilities. They rely on the system to receive alerts when cleaning is required or when any issue is detected. This helps them perform their duties based on actual conditions rather than following fixed schedules.

  The system supports maintenance staff by providing timely notifications and clear information about restroom conditions. This improves response time, reduces unnecessary work, and ensures that hygiene is maintained effectively.

- **Public**

  Public users are individuals who use the restroom facilities. They benefit from improved

cleanliness and better maintained environments as a result of the system's monitoring and alert mechanisms. Their overall experience is enhanced through improved hygiene conditions.

If implemented, the system can also provide users with basic information such as cleanliness status or availability of nearby facilities. This helps users make better decisions and promotes the usage of public sanitation infrastructure.

4.2 Functional Requirements

FR1. **Cleanliness Status Evaluation**

- **Description** : The system shall analyse sensor data to determine the overall cleanliness level of the restroom.
- **Actors** : Municipal Authority
- **Inputs** :
  - –Sensor readings
- **Processing** :
  - –Analyse multiple parameters
  - –Evaluate hygiene level
- **Outputs** :
  - –Cleanliness status displayed
- **Preconditions** :
  - –Valid sensor data must be available
- **Postconditions** :
  - –Cleanliness level is updated

Evaluate Cleanliness

Data Analysis Module

FR2. **Real-Time Monitoring of Restroom Conditions**

- **Description** : The system shall continuously collect real-time data from sensors installed in the restroom to monitor environmental and usage conditions.

•**Actors** : Municipal Authority

•**Inputs** :

–Sensor data (air quality, temperature, humidity, occupancy, water flow)

•**Processing** :

–Collect data from sensors

–Transmit data to server

–Update system in real-time

•**Outputs** :

–Live sensor data displayed on dashboard

•**Preconditions** :

–Sensors must be installed and connected

•**Postconditions** :

–Updated data is available for monitoring

FR3. **Alert Generation**

•**Description** : The system shall generate alerts when abnormal conditions are detected.

•**Actors** : Municipal Authority, Maintenance Staff

•**Inputs** :

–Sensor threshold values exceeded

•**Processing** :

–Compare data with predefined limits

–Trigger alerts

•**Outputs** :

–Alert notifications displayed

•**Preconditions** :

–Threshold values must be defined

•**Postconditions** :

–Alerts are generated for action

FR4. **Occupancy Detection**

- **Description** : The system shall detect user presence and restroom usage using sensors.
- **Actors** : Municipal Authority
- **Inputs** :
  - –PIR sensor data
  - –Door sensor data
- **Processing** :
  - –Detect movement
  - –Track usage patterns
- **Outputs** :
  - –Occupancy status displayed
- **Preconditions** :
  - –Sensors must be active
- **Postconditions** :
  - –Usage data is updated

FR5. **Water Usage and Flush Monitoring**

- **Description** : The system shall monitor water flow to detect usage and flush activity.
- **Actors** : Municipal Authority
- **Inputs** :
  - –Water flow sensor data
- **Processing** :
  - –Analyse flow rate
  - –Detect flushing activity
- **Outputs** :
  - –Water usage status displayed
- **Preconditions** :
  - –Water flow sensor must be functioning

•**Postconditions** :

–Water usage data is updated

FR6. **Dashboard Display**

•**Description** : The system shall provide a web-based interface to display system data and alerts.

•**Actors** : Municipal Authority

•**Inputs** :

–Processed system data

•**Processing** :

–Format and visualize data

•**Outputs** :

–Dashboard with real-time information

•**Preconditions** :

–System must be connected to server

•**Postconditions** :

–Data is visible to users

4.3 Non-Functional Requirements

NFR1. **Performance**

•**Description** : The system shall provide real-time monitoring with minimal delay to ensure timely detection of issues.

•**Constraints** :

–Network connectivity must be stable

–Sensors should respond quickly

•**Acceptance Criteria** :

–Data should be updated within a few seconds

–Alerts should be generated without delay

NFR2. **Reliability**

•**Description** : The system shall operate consistently and provide accurate data without frequent failures.

•**Constraints** :

–Sensors must be properly calibrated

–Hardware components should be maintained

•**Acceptance Criteria** :

–System should run continuously without crashes

–Data accuracy should be maintained

NFR3. **Scalability**

•**Description** : The system shall support expansion to multiple restroom units and locations.

•**Constraints** :

–Backend system must handle increased data load

–Network infrastructure should support scaling

•**Acceptance Criteria** :

–System should handle multiple devices without performance drop

–Data from multiple locations should be managed efficiently

NFR4. **Usability**

•**Description** : The system shall provide a simple and user-friendly interface for easy monitoring.

•**Constraints** :

–Interface design should be clear and minimal

–Navigation should be easy

•**Acceptance Criteria** :

–Users should understand dashboard without training

–Information should be clearly displayed

NFR5. **Security**

•**Description** : The system shall ensure secure access and data protection.

- **Constraints** :
  - –Only authorized users should access the system
  - –Data transmission should be secure
- **Acceptance Criteria** :
  - –Unauthorized access should be prevented
  - –Data should remain protected

Appendix A

Writing an Abstract

Paragraph 1: Background and Problem Statement

- Introduce the **domain or context** of the project.
- Clearly state the **problem or gap** addressed by the project.
- Explain **why the problem is important** in real-world scenarios.
- Keep the explanation **conceptual** and avoid technical details.

Paragraph 2: Objective and Proposed Solution

- State the **main objective(s)** of the project.
- Briefly describe the **system/application developed**.
- Mention the **approach or idea** used to solve the problem.

Paragraph 3: Methodology and Technologies Used

- Specify the **tools and technologies** used (e.g., Python, Java, MySQL).
- Mention any **techniques, algorithms, or frameworks** used.
- Describe the **system design approach** (web app, mobile app, automation, etc.).
- Keep the description **brief but specific**.

Paragraph 4: Results, Outcomes and Conclusion

- Highlight the **key results or outcomes** of the project.
- Mention the **benefits and advantages** of the system.
- State possible **applications or use cases**.
- Conclude with the **overall impact or effectiveness**.

General Writing Rules

- Total length should be **200–300 words**.
- Use **past tense** since the project is completed.
- Follow **third person writing** (avoid "I" and "we").
- Maintain **clear, concise, and formal academic language**.
- Avoid **citations, diagrams, equations, and code**.

Appendix B

Writing Introduction

General Structure of Introduction

- The Introduction chapter should clearly present the **context, motivation, and direction** of the project.
- It must be structured into logical sections such as **Problem Scenario, Proposed Solution, Purpose, and Scope**.
- The content should move from **general problem understanding to specific system contribution**.
- Avoid unnecessary technical depth; focus on **clarity and justification**.

Problem Scenario

- Describe the **existing system or current practice**.
- Identify key **problems, inefficiencies, or limitations**.
- Highlight **real-world challenges** such as delays, errors, or manual effort.
- Justify the **need for improvement or automation**.
- Use simple, clear language to explain the **impact of the problem**.

Proposed Solution

- Clearly present the **system or application developed**.
- Explain how the proposed system **addresses the identified problem**.
- Mention the **core idea or working principle** (e.g., automation, rule-based logic).
- Briefly include **technologies used** (without going into deep technical details).
- Emphasize **improvements over existing systems**.

Purpose

- State the **main aim of the project**.
- Explain the **expected outcomes or goals**.
- Highlight benefits such as **efficiency, accuracy, transparency, and scalability**.
- Show how the system contributes to **solving real-world problems**.

Scope

- Define the **boundaries of the project**.
- Specify **where and how the system can be used**.
- Mention key **features and functionalities covered**.
- Clarify what is **included and not included** in the project.
- Indicate potential **future enhancements or extensions**.

Writing Guidelines

- Maintain a **logical flow** from problem to solution.
- Use **formal academic language**.
- Avoid repetition and overly technical explanations.
- Ensure each section is **clear, concise, and well-connected**.
- Use **third person narration** and consistent tense.

Appendix C

Writing Literature Survey

General Structure of Literature Survey

- The Literature Survey chapter should present a **systematic review of existing work** related to the project.
- It must demonstrate **understanding of the domain, prior research, and existing solutions**.
- The chapter should be logically organized into sections such as **Domain Survey, Research Review, Summary, Research Gaps, and Comparative Study**.
- The content should move from **general domain understanding to specific research analysis**.

Domain Survey

- Provide an **introduction to the domain** (e.g., EdTech, Scheduling Systems).
- Explain the **importance and relevance** of the domain.
- Describe **existing practices and their limitations**.
- Introduce **automation or modern approaches** used in the domain.

Core Terminologies and Definitions

- Define important **technical terms used in the domain**.
- Ensure definitions are **clear, concise, and relevant**.
- Use consistent terminology throughout the report.

Associated Theoretical Concepts

- Present key **theoretical foundations** (e.g., CSP, algorithms, load balancing).
- Explain concepts in **simple and understandable terms**.
- Relate each concept to **its role in the project**.

Challenges and Open Problems

- Identify **current challenges in the domain**.
- Highlight issues such as **scalability, fairness, flexibility, and adaptability**.
- Explain why these challenges **require improved solutions**.

Research Review

- Include **multiple relevant research papers** (5–10 recommended).
- For each paper, present:
  - Abstracted Summary
  - Problem Statement
  - Proposed Solution
  - Methodology
  - Results
  - Conclusion
- Ensure proper **citation and referencing**.
- Avoid copying; write in **your own words**.
- https://www.mybib.com/tools/ieee-citation-generator : helps you to generate content similar to sample listed in the document

Summary of Reviewed Papers

- Present a **tabular comparison of all reviewed papers**.
- Include columns such as:
  - Paper Title
  - Authors and Year
  - Problem Statement
  - Methodology
  - Key Findings
- Ensure the table is **clear, compact, and readable**.

Identified Research Gaps

- Identify **limitations in existing research**.
- Highlight missing features such as:
  - Lack of real-time adaptability
  - Poor scalability
  - Limited fairness considerations
  - Lack of system integration
- Clearly show how these gaps **justify your proposed system**.

Comparative Study of Existing Applications

- Compare **existing systems or applications**.
- Use a table with columns such as:
  - Application Name
  - Features
  - Performance
  - User Experience
  - Limitations
- Highlight how your system is **different or improved**.

Writing Guidelines

- Maintain **logical flow and coherence**.
- Use **formal academic language**.
- Avoid plagiarism; always **paraphrase properly**.
- Ensure proper **referencing and citations**.
- Keep explanations **clear, structured, and relevant**.

Appendix D

Hardware and Software Requirements

General Structure

- This chapter should clearly describe the **hardware and software environment** required for the project.
- The content must be organized into sections such as **Introduction, Hardware Requirements, and Software Requirements**.
- The focus should be on **justifying the chosen technologies** based on performance, scalability, and compatibility.
- Avoid unnecessary theory; keep the content **practical and specification-oriented**.

Introduction

- Provide a brief overview of the **tools and technologies used**.
- Mention the **purpose of this chapter**.
- Highlight criteria such as **performance, scalability, compatibility, and efficiency**.

Hardware Requirements

- Describe the **physical system requirements** needed to develop and run the project.
- Include both **minimum** and **recommended specifications**.
- Ensure specifications are **realistic and relevant** to the project.

Minimum Hardware Specifications

- Provide a table listing:
  - Processor
  - RAM
  - Storage
- Mention the **basic configuration required** to run the system.
- Ensure the system can function under **minimal conditions**.

Recommended Hardware Specifications

- Provide a table listing:
  - Processor
  - RAM
  - Storage
- Mention configurations for **optimal performance**.
- Highlight improvements over minimum specifications.

Software Requirements

- Describe all **software components used in the project**.
- Categorize into:
  - Operating Systems
  - Development Tools
  - Programming Languages and Frameworks

Operating System

- List supported **operating systems and versions**.
- Ensure **cross-platform compatibility** if applicable.
- Present information in a **tabular format**.

Development Tools

- Mention tools used for:
  - Coding (IDE/Editor)
  - Version control
  - Collaboration
- Provide **purpose for each tool**.
- Keep descriptions **short and clear**.

Programming Languages and Frameworks

- List all **technologies used in development**.
- Specify their **role in the system** (frontend, backend, database).
- Present details in a **table format**.
- Ensure alignment with the **actual implementation**.

Writing Guidelines

- Use **clear and concise technical language**.
- Maintain **consistency in tables and formatting**.
- Ensure all specifications are **accurate and justified**.
- Avoid unnecessary explanations; focus on **relevance**.
- Keep the structure **well-organized and easy to read**.

Appendix E

Writing Software Requirements Specification

General Structure

- The Software Requirements Specification (SRS) defines the **functional and non-functional requirements** of the system.
- It should clearly describe **what the system does** and **how it behaves**.
- The chapter must be structured into:
  - Users
  - Functional Requirements
  - Non-Functional Requirements
- Avoid implementation details; focus on **system behavior and expectations**.

Users

- Identify all **types of users interacting with the system**.
- For each user:
  - Describe their **role and responsibilities**.
  - Explain how they **interact with the system**.
  - Highlight their **permissions and access levels**.
- Use clear categories such as:
  - Administrator
  - Faculty/User
  - Coordinator/Manager
- Ensure descriptions are **detailed but easy to understand**.

Functional Requirements

- List all **core functionalities of the system**.
- Each requirement should:
  - Start with a **clear action statement** (e.g., "The system should...").
  - Describe **what the system must do**.
  - Be **specific, measurable, and testable**.
- Cover all major operations such as:
  - Automation processes
  - Data management
  - User interactions
  - Notifications and reporting
- Ensure requirements are:
  - **Complete** (no missing functionality)
  - **Consistent** (no contradictions)
  - **Unambiguous**

Non-Functional Requirements

- Define the **quality attributes of the system**.
- Focus on how the system performs rather than what it does.
- Include aspects such as:
  - **Security** – Authentication, data protection
  - **Usability** – Ease of use, user interface design
  - **Performance** – Speed, responsiveness
  - **Reliability** – System stability and error handling
  - **Scalability** – Ability to handle increased load
- Each requirement should be **clear and verifiable**.

Writing Guidelines

- Use **formal and precise language**.
- Avoid vague terms like "fast" or "user-friendly" without clarification.
- Maintain **consistency in terminology**.
- Ensure all requirements are **logically organized**.
- Keep descriptions **clear, structured, and testable**.

Special Note: Requirement Traceability and Connectivity

- Every **Functional Requirement (FR)** must be clearly linked to:
  - **Use Case Diagrams and Descriptions**
  - **System Design Components (Architecture, Modules)**
  - **Algorithms and Logic Implementation**
  - **Test Cases and Validation Scenarios**
- Ensure **end-to-end traceability**, where each requirement can be tracked from:

$$\textit{Requirement} \rightarrow \textit{Design} \rightarrow \textit{Implementation} \rightarrow \textit{Testing}$$

- Each functional requirement should:
  - Appear in at least one **use case**
  - Be implemented through a **specific module or algorithm**
  - Be validated using **test cases**
- Missing linkage indicates:
  - Incomplete design
  - Unimplemented functionality
  - Untested system behavior
- Maintaining this connectivity ensures:
  - **Consistency across all chapters**
  - **Better evaluation during project review**
  - **Higher academic quality and completeness**

RTM for personal reference

Req ID	Requirement	Use Case	Module	Algorithm	Test Case
FR1	Auto Scheduling	Generate Schedule	Scheduler	Constraint-Based / Greedy	TC1
FR2	Duty Allocation	Assign Duties	Allocation Engine	Load Balancing / Heuristic	TC2
FR3	Manual Override	Modify Duties	Admin Module	Rule-based Update	TC3
FR7	Duty Swap	Request Swap	Swap Module	Matching Logic	TC7

Appendix F

Writing System Design

General Structure

- The System Design chapter describes **how the system is structured and implemented**.
- It should translate **requirements into design components**.
- The chapter must include:
  - Architecture Diagram
  - Data Flow Diagrams (DFD)
- Focus on **system organization, modules, and data movement**.

Architecture Diagram

- Present a **high-level structure of the system**.
- Use a **layered or modular approach** (e.g., Presentation, Business, Service, Data layers).
- Clearly show:
  - User interfaces
  - Core processing modules
  - Backend services
  - Database interactions
- Ensure diagram is:
  - **Clean and readable**
  - Properly **labeled**
  - Consistent with implementation

Architecture Description

- Explain each layer or module in **paragraph form**.
- Typical layers include:

  - **Presentation Layer** – User interface and interaction
  - **Business Layer** – Core logic and decision making
  - **Service Layer** – Functional services and APIs
  - **Data Layer** – Database and storage
- Describe:
  - Roles of each layer
  - Interaction between layers
  - Responsibilities of components
- Ensure explanation aligns with **functional requirements**.

Data Flow Diagram (DFD)

- Illustrate how **data moves through the system**.
- Start with a **Context Diagram (Level 0)**.
- Show:
  - External entities (Users)
  - System boundary
  - Data inputs and outputs

Context Diagram Guidelines

- Represent the system as a **single process**.
- Identify all **external actors** (Admin, Faculty, Coordinator).
- Show **data flow between users and system**.
- Avoid internal details; keep it **high-level**.

DFD Levels (Optional Enhancement)

- Level 1 DFD can be used to show:
  - Scheduling process
  - Duty allocation process
  - Notification handling
- Break system into **sub-processes**.

Diagram Writing Guidelines

- Use **consistent symbols** (process, data store, entity, flow).
- Ensure diagrams are **neat and properly aligned**.
- Add **captions and labels** for all diagrams.
- Refer to diagrams in the **text explanation**.

Special Note: Design Traceability

- Every design component must map to **Functional Requirements (FRs)**.
- Ensure clear linkage:

$$\textit{Functional Requirement} \rightarrow \textit{Use Case} \rightarrow \textit{Architecture Module} \rightarrow \textit{Implementation}$$

- Each module in the architecture should:
  - Implement one or more **functional requirements**
  - Be represented in **use case diagrams**
  - Be validated through **test cases**
- Data Flow Diagrams should reflect:
  - Actual system operations
  - Real data interactions defined in SRS
- Lack of traceability indicates:
  - Missing implementation
  - Incomplete design
  - Weak system validation
- Maintaining this connectivity ensures:
  - **Consistency across chapters**
  - **Stronger project evaluation**
  - **Well-engineered system design**

Appendix G

Writing Detailed Design

Purpose of Detailed Design

- The Detailed Design chapter explains **how the system is internally structured**.
- It converts:

$$\textit{System Design} \rightarrow \textit{Implementable Components}$$

- It must clearly show:
  - User interactions
  - Workflow logic
  - Data structures

Use Case Diagram

- Represents **interaction between users and system functionalities**.
- Identify all **actors**:
  - Administrator / COE
  - Faculty
  - Department Coordinator
- Include all **major use cases**:
  - Login / Authentication
  - Schedule Exams
  - Allocate Duties
  - Request Duty Swap
  - Approve / Reject Swap
  - Notifications
- Diagram must:
  - Clearly show actor–system relationships
  - Use standard UML notation
  - Be consistent with Functional Requirements

Use Case Description Guidelines

- Each use case should map to a **Functional Requirement (FR)**.
- Include:
  - Actor
  - Description
  - Inputs / Outputs
  - Flow of events

Activity Diagram

- Represents **step-by-step workflow of the system**.
- Show:
  - Decision points (if/else)
  - Sequential steps
  - Parallel processes (if any)
- Must include:
  - Start and End nodes
  - Actions (login, scheduling, swap request)
  - Decision nodes (approve/reject)
- Ensure:
  - Logical flow is clear
  - No missing transitions

Document Structure (Database Design)

- Define **data storage format** (e.g., MongoDB documents).
- Each structure must include:
  - Field names
  - Data types
  - Purpose of each field
- Typical collections include:
  - Users
  - Faculty Availability
  - Faculty Duties
  - Exam Dates
  - Student Details
  - Backlog Information

- Ensure:
  - Proper naming conventions
  - Avoid redundancy
  - Data consistency

Document Explanation Guidelines

- After each structure:
  - Explain purpose of the document
  - Describe how it supports system functionality
  - Mention how it interacts with other documents
- Example:
  - User document → Authentication + Role Management
  - Faculty Duties → Duty allocation tracking
  - Availability → Scheduling constraints

Design Best Practices

- Keep design **modular and scalable**
- Ensure **clear separation of concerns**
- Maintain **data integrity**
- Avoid duplication of data
- Use **consistent naming conventions**

Special Note: End-to-End Traceability

- Ensure strong linkage across:

$$\textit{Functional Requirements} \rightarrow \textit{Use Cases} \rightarrow \textit{Activity Diagram} \rightarrow \textit{Database Design} \rightarrow \textit{Implementation} \rightarrow \textit{Test Cases}$$

- Every:
  - Use Case must come from an FR
  - Activity must reflect a Use Case flow
  - Data structure must support operations in activity
- Missing linkage indicates:
  - Incomplete design
  - Poor implementation clarity
- Strong traceability ensures:
  - High evaluation score
  - Industry-level documentation
  - Easy validation and testing